

git Basics

FRC Team 11025 – Garage Gang

Presented by: Nat Sweeney

What is Version Control?

Instead of a directory filled with files titled “something_v2”, “something_final”, and “something_final_working_real”, version control lets you have a history of your code without extra files.

Version control also helps with collaborative environments between programmers instead of needing a server containing all files or a long email chain with code changes.

Why Use Version Control?

Keep track of changes in code over time.

Manage collaboration between team.

Remove the need for centralized storage and workspace.

Allows single user to work on different computers.

We use git for our version control and utilize GitHub for managing our various repositories.

Basic git Commands and Terms

add : Selects file(s) that you want to save version of.

commit : Saves previously added files with message detailing what changes were made.

push : Make your changes part of the version history of the code.

pull : Get the most recent version and recent changes to code.

branch : Create a branch for a specific purpose.

checkout : Switch to a different branch

Staging Area : Where a modified file is before being pushed to the current branch.

How to Use git (Heavily Simplified)

1. First **checkout** to a **branch** for an issue and **pull** the most recent changes.
2. Make a noteworthy change and test that the change works.
3. After testing, **add** the modified file to the **staging area**, **commit** the changes with a descriptive message, and **push** to the branch.
4. Once finished with a specific issue, create a **merge** request and have someone else review it.

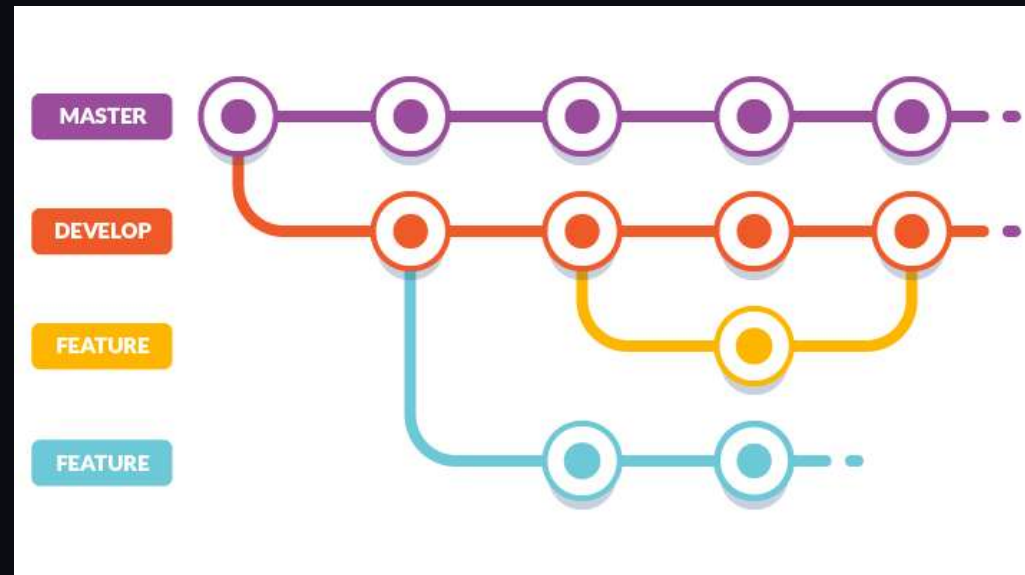
Branches

One fundamental concept with git is branches.

Branches have various benefits:

- Code organization

- Parallel development



Merging vs Rebasing

Once a branch has been completed (issue fixed, feature added, event finished, etc.) it needs to be recombined with its head branch.

The two methods of recombining are merging and rebasing:

Merge: Preserves full commit history and timeline.

Rebase: Creates linear sequence of events.

Before Integration (Common starting point):

```
      C---D (feature-branch)
      /
A---B---E---F (main)
```

After Merge:

```
      C---D
      /   \
A---B---E---F---M (main)
```

M is a merge commit with two parents: F and D

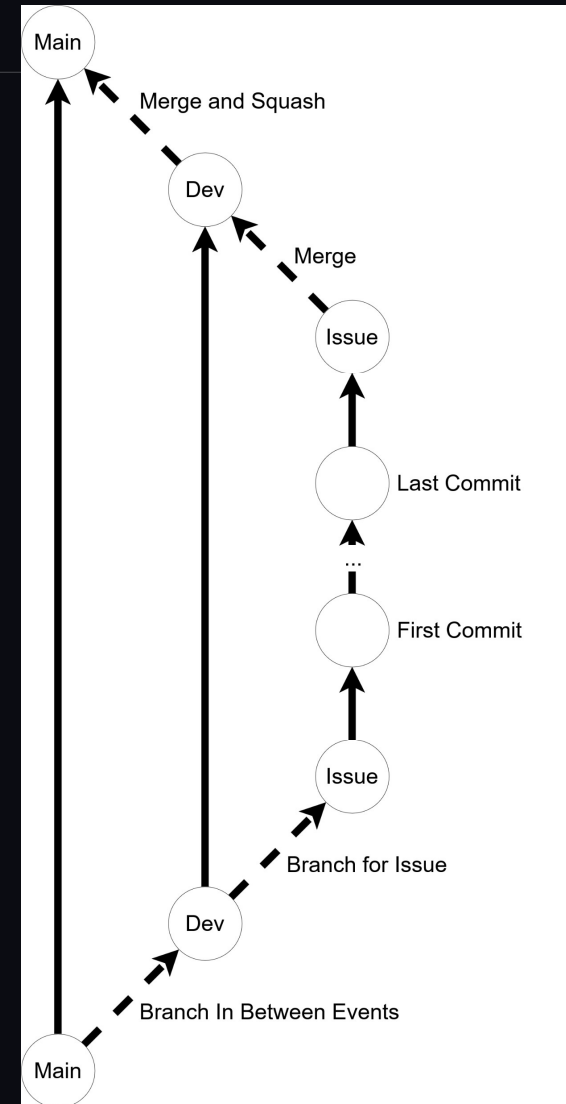
After Rebase:

```
A---B---E---F---C'---D' (main)
```

When to Merge

Merge when combining a new feature or bug fix into a primary branch (main or development).

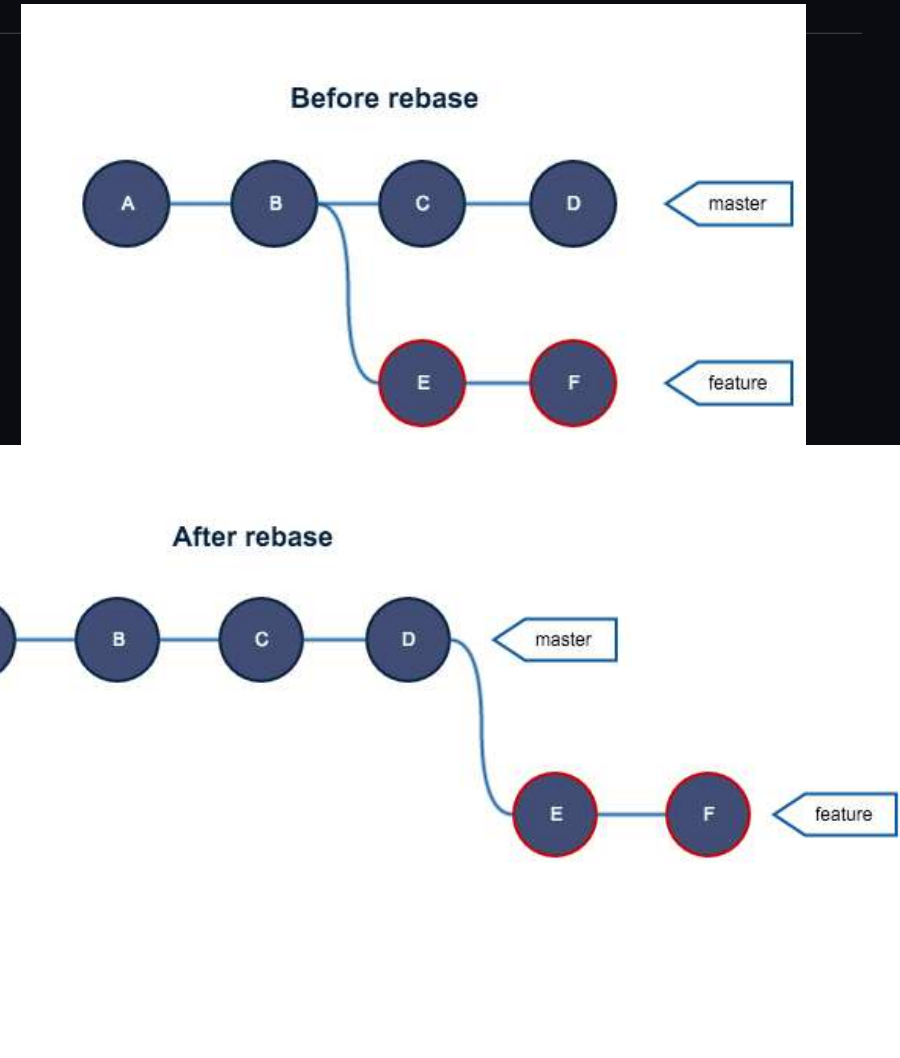
Example: John created a branch to fix a motor issue. Once they are done and have tested that it works, they merge their branch into the development branch.



When to Rebase

Rebase when working on a secondary branch that someone else has also worked on.

Example: John wrote some code on a branch, but wants to include some code Jane wrote on the head branch after he branched off his work. To do this he rebased his branch.



git Command Line Workflow

1. Switch to development branch.
2. Pull most recent changes.
3. Modify files and test changes.
4. Commit and push changes.
5. When feature is fully developed, merge the branch.

```
MINGW64:/c/Users/natbs/Documents/frc/structuredLearningResources

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ ls
README.md  battleship/  hands-on-git-demo/  numberGuesser/

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ git pull
From https://github.com/garage-gang/structuredLearningResources
   27131e0..3550235  git-learning -> origin/git-learning
Already up to date.

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ vim hands-on-git-demo/instructions.md

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ git add hands-on-git-demo/instructions.md

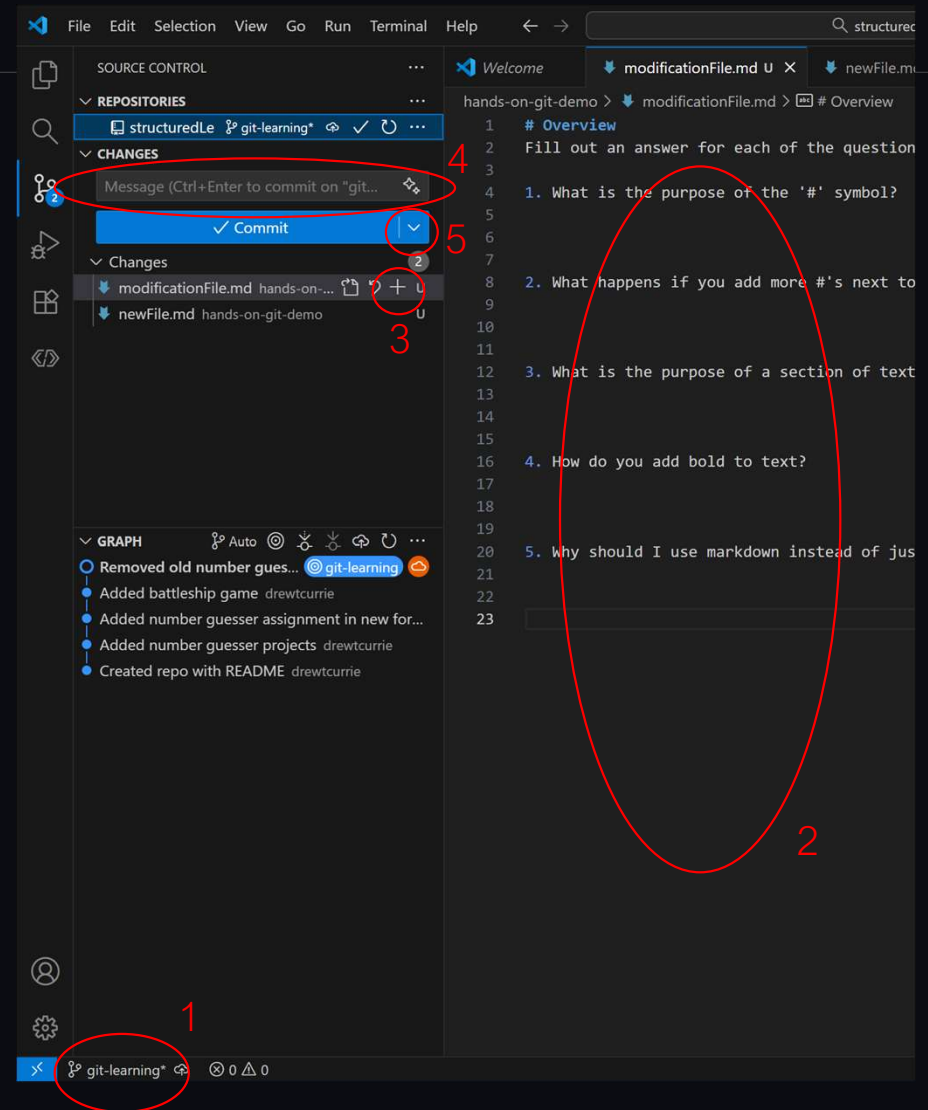
natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ git commit -m 'removed unnecessary wording from step 3'
[main 73542d4] removed unnecessary wording from step 3
   1 file changed, 1 insertion(+), 1 deletion(-)

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 22 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 361 bytes | 361.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/garage-gang/structuredLearningResources.git
   f9b9aff..73542d4  main -> main

natbs@nattop MINGW64 ~/Documents/frc/structuredLearningResources (main)
$ |
```

git VSCode Workflow

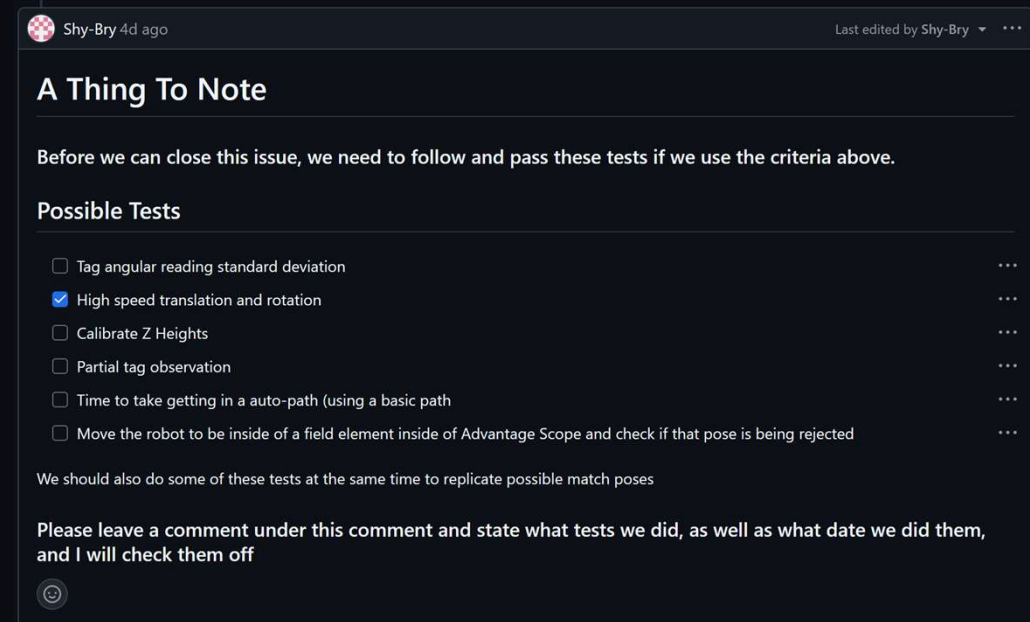
1. Swap to the development branch.
2. Make changes to files.
3. In the 'Source Control' tab, click the + next to your modified file(s).
4. Write a meaningful commit message.
5. Commit and push staged files.



Aside: Markdown Files

Markdown is a file format that can create nicely formatted files, which are especially useful when it comes to git.

Issues and Merge Requests in GitHub heavily benefit from nicely formatted and informative Markdown text.



Hands-On Example

Instructions:

1. Fork the 'structuredLearningResources' repo from the organization.
2. Clone the forked repo and create your own branch with your name off of the 'git-learning' branch.
3. Using the link in instructions.md, create a nicely formatted Markdown file to explain what a section of code you've recently written does.

If you haven't written or don't have access to any robot code, make a nicely formatted recipe for your favorite food.

Name the Markdown file something unique (like your name) to avoid any conflicts (which we want to avoid right now).

4. In GitHub, create a merge request and assign someone else to review your Markdown file, give feedback, and then approve the merge request.

Additional Resources

[Interactive git Tutorial and Lessons](#)

[git Documentation](#)